

Comportamento polimórfico

Ficha Padrão de Entrega de Atividade Semanal - versão final

1. Identificação

Atividade: A9 - Comportamento polimórfico. Etapa: Entrega final. Avaliação vinculada: AP2.

Equipe: TechFix. Integrantes: Arthur Scharfenberger e Lucas Oliveira da Silva.

Data de preparação e verificação: 04/10/2026. A data de submissão será a registrada no AVA.

Repositório: <https://github.com/ArthurScharfenberger/TechFix-Assistencia-Tecnica>

Referência de entrega: branch main. Tag: não utilizada nesta entrega.

2. Conteúdo da atividade

Ponto de variação e solução integrada

O roteiro inicial de diagnóstico varia conforme o equipamento recebido pela assistência. Um notebook exige verificações de fonte, memória, armazenamento e refrigeração; um celular exige verificações de bateria, tela e slots de chip. Centralizar essas diferenças na ordem de serviço exigiria selecionar o tipo do equipamento para decidir quais etapas apresentar.

O incremento introduz a interface `RoteiroDiagnostico`, implementada pela hierarquia `Equipamento`. `Notebook` e `Celular` fornecem as duas implementações concretas. `OrdemServico` consulta o contrato e apresenta o roteiro junto aos dados do atendimento. `Main` demonstra tanto essa integração quanto uma coleção declarada pelo tipo da interface.

O roteiro orienta o técnico; não executa testes físicos nem atesta o defeito. As etapas são decisões de modelagem acadêmica do incremento. A integração ocorre no programa Java existente; não há comunicação com o frontend TypeScript, API ou persistência nesta entrega.

Contrato documentado em Javadoc

Arquivo: `backend/src/main/java/RoteiroDiagnostico.java`.

```
public interface RoteiroDiagnostico {  
    List<String> gerarRoteiroDiagnostico();  
}
```

O Javadoc exige uma lista imutável, não vazia, sem elementos nulos ou em branco, ordenada pela sequência recomendada. A consulta não modifica o equipamento e nunca retorna null. O método não recebe parâmetros: usa a configuração validada no construtor de cada equipamento. A implementação utiliza `List.of`, e os testes verificam os compromissos do contrato.

`Equipamento` é abstrata e declara `implements RoteiroDiagnostico`. Ela mantém os dados e validações comuns; suas subclasses concretas devem implementar o método do contrato. A descrição de atendimento da Atividade 8 permanece disponível.

Duas implementações com lógicas distintas

Notebook.gerarRoteiroDiagnostico retorna, nesta ordem: verificar fonte e conector de alimentação; testar a quantidade de memória RAM cadastrada; verificar armazenamento e refrigeração. Para um notebook de 16 GB, a segunda etapa é exatamente "Testar os 16 GB de memória RAM". A memória precisa ser positiva, conforme a validação existente.

Celular.gerarRoteiroDiagnostico retorna: verificar bateria e conector de carga; testar tela e resposta ao toque; testar um slot ou os dois slots de chip, conforme dualChip. O condicional local trata a configuração do celular, sem verificar a classe do objeto. As implementações diferem nos procedimentos e nos dados que determinam o roteiro.

Código cliente usando o tipo abstrato

Trecho integrado em OrdemServico.java:

```
public List<String> getRoteiroDiagnostico() {  
    RoteiroDiagnostico roteiro = equipamento;  
    return roteiro.gerarRoteiroDiagnostico();  
}
```

exibirOrdemServico chama esse método e imprime as etapas. O cliente não usa instanceof, casts, getClass ou switch por tipo. A criação do objeto define a implementação, e a ligação dinâmica seleciona o método no momento da chamada.

Trecho executável de Main.java:

```
List<RoteiroDiagnostico> roteiros = List.of(notebook, celular);  
for (RoteiroDiagnostico roteiro : roteiros) {  
    roteiro.gerarRoteiroDiagnostico().forEach(System.out::println);  
}
```

Por que é polimorfismo

A mesma chamada gerarRoteiroDiagnostico, feita por uma referência RoteiroDiagnostico, executa procedimentos diferentes conforme o objeto concreto. Isso demonstra substituição pelo contrato e ligação dinâmica, além do reaproveitamento de atributos e métodos proporcionado pela herança.

Para adicionar outro equipamento, uma nova subclasse fornece o roteiro e respeita o contrato. O código cliente da ordem de serviço não precisa acrescentar verificações de tipo. A construção e o cadastro de novos tipos continuam sendo responsabilidades separadas.

Testes obrigatórios e evidências

Comandos executados na pasta backend em 04/10/2026:

```
mvn test
java -cp target/classes Main
```

Resultado Maven: BUILD SUCCESS. 14 testes executados, 0 falhas, 0 erros e 0 ignorados.

EquipamentoTest: 4 testes; OrdemServicoTest: 4 testes; RoteiroDiagnosticoTest: 6 testes. Os oito testes anteriores continuam passando.

Os seis novos testes em backend/src/test/java/RoteiroDiagnosticoTest.java verificam:

- notebookRetornaEtapasConhecidasNaOrdem: compara a lista completa para um notebook de 16 GB com três valores esperados explícitos.
- celularDualChipRetornaEtapasConhecidasNaOrdem: compara a lista completa para um celular dual chip com três valores esperados explícitos.
- celularSimplesOrientaApenasUmSlot: verifica a alternativa de um único slot.
- mesmaReferenciaAbstrataDespachaParaImplementacoesDiferentes: percorre List<RoteiroDiagnostico> e compara os resultados específicos das duas implementações.
- contratoMantemListaImutavelEConsultaSemEfeitosColaterais: verifica conteúdo válido, rejeição de alteração da lista, repetibilidade e preservação dos dados para ambos os equipamentos.
- ordemServicoIntegraAmbosOsRoteirosSemAlterarStatus: verifica a delegação nos dois casos e a manutenção do status ABERTA.

Trecho da saída do Main, após os dados do equipamento da OS:

```
ROTEIRO DE DIAGNÓSTICO
- Verificar fonte e conector de alimentação
- Testar os 16 GB de memória RAM
- Verificar armazenamento e refrigeração
```

A demonstração pela coleção também apresenta bateria e carga, tela e toque e os dois slots para o celular Samsung. As saídas são conferidas por valores conhecidos nos testes; não dependem de entrada interativa ou de serviços externos.

Rastreabilidade dos critérios da entrega

- Abstração expressiva e Javadoc: RoteiroDiagnostico.java.
- Duas implementações distintas: Notebook.java e Celular.java.
- Cliente com referência abstrata: OrdemServico.getRoteiroDiagnostico e coleção em Main.
- Justificativa: seção Por que é polimorfismo desta ficha e README do backend.
- Testes por implementação e pelo contrato: RoteiroDiagnosticoTest.java.
- Integração e regressão: impressão da OS no Main e suíte Maven completa.

3. Decisões e pendências

Foi escolhida uma interface de comportamento para tornar explícito o ponto de variação. A hierarquia anterior foi aproveitada para preservar a compatibilidade com o modelo existente. As etapas usam listas imutáveis para evitar que o consumidor altere o roteiro retornado.

Os requisitos de código e testes desta entrega foram implementados e verificados. A equipe deve ler a ficha e conferir o conteúdo antes da submissão no AVA. Não se declara que houve entrega de esboço, assinatura ou validação prévia dos integrantes. Esta atividade possui apenas a versão final preparada nesta alteração.

4. Declaração de uso de Inteligência Artificial

(X) Foi utilizada ferramenta de IA nesta atividade. Ferramenta: OpenAI Codex. Data: 04/10/2026.

Objetivo e resumo da interação: analisar os enunciados fornecidos, completar o incremento de comportamento polimórfico no projeto, implementar contrato e duas lógicas concretas, integrar a ordem de serviço, criar e executar testes e preparar somente a documentação final, seguida de commit e push.

Resultado aproveitado: implementação de RoteiroDiagnostico, métodos de Notebook e Celular, integração em OrdemServico e Main, seis novos testes JUnit, documentação e geração desta ficha. A IA participou diretamente da produção do código e do texto, além da revisão.

Verificação realizada durante a preparação: execução de mvn test, execução do Main, conferência do diff e revisão visual do PDF. Registro complementar: DEVLOG.md e backend/IA.md. A verificação automatizada não representa aprovação pessoal dos integrantes.

5. Responsabilidade e autoria

Arthur Scharfenberger e Lucas Oliveira da Silva são os integrantes identificados no projeto e responsáveis pela revisão e apresentação da entrega. A confirmação de leitura, compreensão e autoria deve ser feita por eles no momento da submissão. Nenhuma assinatura foi aplicada automaticamente.

Declaração para validação dos integrantes: ao submeter esta ficha, a equipe confirma que leu e compreendeu o conteúdo, assume responsabilidade por sua correção e consegue explicar o contrato, as implementações e os testes, inclusive as partes produzidas com apoio de IA.